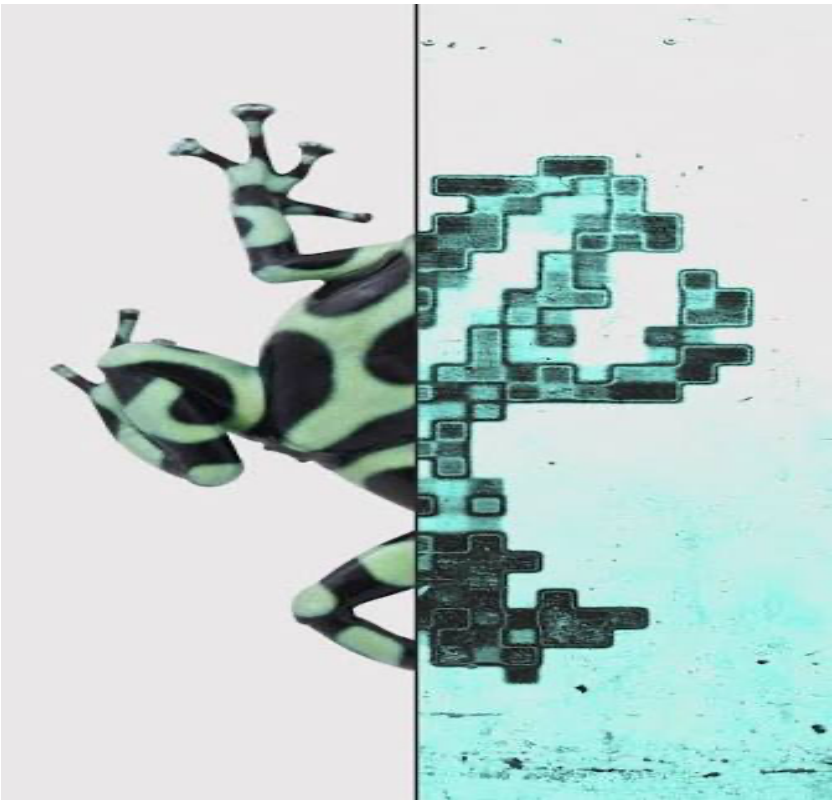
Pyflate

Benchmark Analysis, Software Optimization and Hardware Acceleration

Ori Daniel & Or Sasson

Hardware/Software Project

September 2026



1. What is the Pyflate Benchmark?

- Pyflate is a benchmark that tests the speed of the bzip2 decompression algorithm.
- It is implemented in pure Python, with no calls to fast C libraries.
- The test consists of decompressing a file using a Python implementation of bzip2 decompression.
- It measures how fast a Python version can execute the math and logic operations of decompression without external C-based shortcuts.

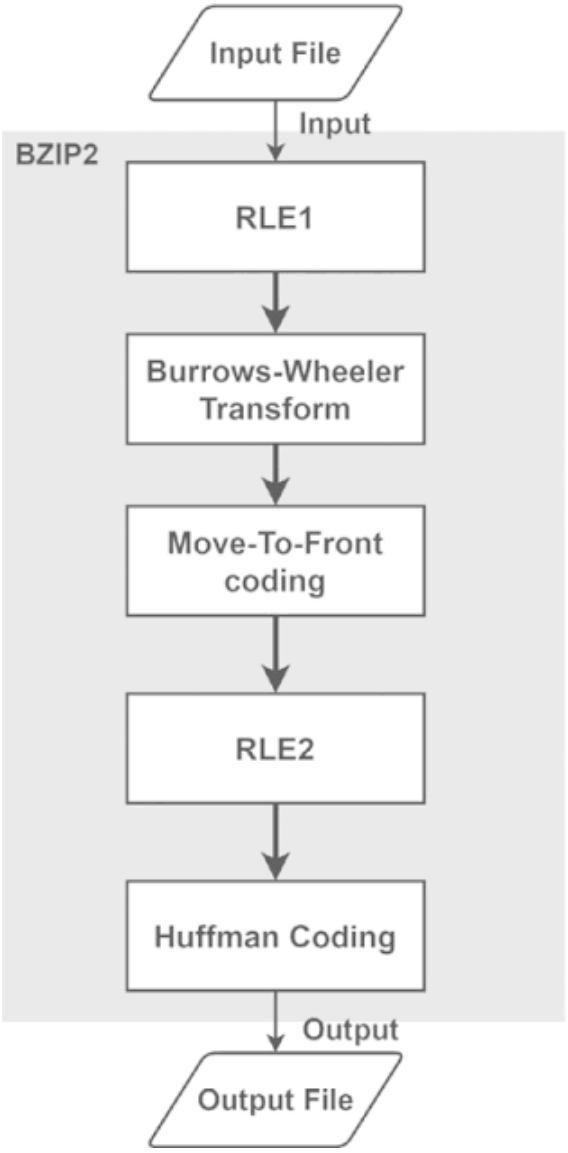
The workload is a 67.5 KB bzip2 archive that expands to 400 KB. One decompression decodes 148,271 Huffman symbols.

2. Implementation Overview — the bzip2 Algorithm

bzip2 is a file format and a file compression algorithm. It uses the Burrows-Wheeler algorithm, invented by David Wheeler and published by Michael Burrows in 1994. It is a five-stage algorithm that runs on small blocks of a file.

2.1 The five stages

- **Run-Length Encoding, pass 1:** replaces a sequence of 4–255 identical consecutive bytes with a single character and a count. Prevents repetitive data from degrading the BWT sorting phase.
- **Burrows-Wheeler Transform (BWT):** reorders the block to group similar characters together.
- **Move-to-Front (MTF) encoding:** turns repeated characters into zeros, which preserves the meaning of a repeated character while helping Huffman coding.
- **Run-Length Encoding, pass 2:** targets the large number of zeros generated by the previous stage.
- **Huffman coding:** encodes characters by frequency. High-frequency characters receive short bit sequences.



2.2 Run-Length Encoding

RLE compresses long repeating strings of the same character. It outputs the first four instances of a character and then represents the rest of its appearances with a count:

```
aaaaaaaaaa  ->  aaaa6
```

2.3 Burrows-Wheeler Transform

The BWT is the most crucial stage of the algorithm. The idea is to rearrange a block of data into runs of identical characters without compressing it or losing any information. The resulting column is the transformed version of the string, which is used to restore the original during decompression.

Encoding, using the example string PEPPER

- Add $ to the end of the string: PEPPER$
- Generate all N cyclic shifts.
- Sort the circular shifts.
- Take the last column — that is the BWT string: RPP$PEE

Decoding, from the BWT string RPP$PEE

Run N times:

- Add the transformed string as a new first column to the table.
- Sort the table.
- Combine the columns.

The decoded string is the row with $ at the end: PEPPER$

Why does this help?

Sorted data is easily compressible. The Burrows-Wheeler Transform exploits the local context of real-world data: in real languages and structured data, specific characters precede certain sequences — such as the letter ‘t’ before ‘he’ in the word ‘the’.

2.4 Huffman Encoding

Huffman coding takes a string with a fixed alphabet and compresses it based on the frequency of those characters. The algorithm uses a tree and a greedy assumption to give each character a prefix-free encoding, so the codes can be distinguished inside a contiguous bit string.

For example, in the string PPPPPPPPPABCD there are five distinct characters. It is most efficient to give ‘P’ the shortest encoding:

```
   P: 0      A: 100     B: 101     C: 110     D: 111


PPPPPPPPPABCD  ->  00000000000100101110111  =  0x000977
```

2.5 Move-to-Front

Move-to-front takes consecutive repeated letters and turns all but the first appearance into ‘0’. A zero indicates a “duplicate the letter before me” operation. This makes sense in context: the BWT produces a high frequency of consecutive identical letters, and after this manipulation a “repeated letter” becomes its own symbol — exactly what the following Huffman stage exploits. The second RLE pass then compresses the resulting runs of zeros.

3. Initial Analysis

3.1 Environment and methodology

All measurements in this report were taken on the course QEMU virtual machine, so every number is comparable against every other number.

Item	Value
Host	QEMU/KVM virtual machine
CPU	Intel Xeon E5-2630 v3 @ 2.40 GHz, 1 logical CPU
Kernel	Linux 5.15.0-1106-kvm x86_64, glibc 2.35
Python	3.10.12 (64-bit)
pyperformance	1.14.0
Profilers	py-spy 0.4.1, perf, python3-dbg
Workload	interpreter.tar.bz2 — 67.5 KB in, 399,360 bytes out

Everything below is reproduced by a single script, Pyflate/script_pyflate.sh, which sets up the environment, runs the baseline, runs the optimized version, and writes every artefact referenced in this report into Pyflate/results/.

3.2 Baseline results

We ran:

```
pyperformance run --python /usr/bin/python3.10 --bench pyflate
```

60 decompressions of a 67.5 KB bzip2 archive (399 KB decompressed), across 20 different processes. These were the results:

Metric	Value
Mean	1.124 s ± 0.018 s
Min / Max	1.108 s / 1.229 s
Samples	60 (20 processes × 3 values)

The hardware counters for the same workload, five decompressions repeated three times:

```
perf stat -r 3 -e cycles,instructions,branches,branch-misses


  13595107086      cycles                                ( +-  0.28% )
  35899254692      instructions   # 2.66 insn per cycle ( +-  0.11% )
   7151658799      branches                              ( +-  0.13% )
     24879633      branch-misses  # 0.35% of branches   ( +-  3.33% )


       5.6938 +- 0.0147 seconds time elapsed  ( +-  0.26% )
```

results/baseline/pyflate_perfstat_baseline.txt

3.3 Profiling with py-spy

We used py-spy to profile the CPython benchmark. To produce the flame graph we ran:

```
py-spy record --rate 999 --output results/baseline/flamegraph.svg
```

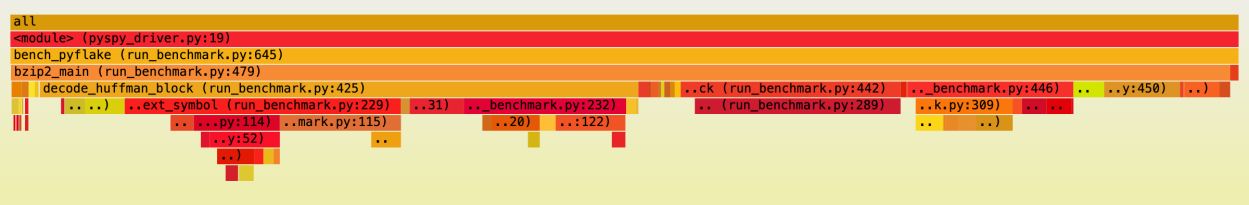


Figure 1 — Baseline flame graph.

The flame graph shows the problem decode_huffman_block occupies the whole program.. For that we merged all lines of each function from the folded stacks. **Cumulative** counts a frame anywhere in the stack; self counts only where the CPU actually was:

Function	Cumulative	Self
decode_huffman_block	99.06%	21.05%
find_next_symbol	46.43%	17.16%
snoopbits	17.72%	9.42%
move_to_front	16.92%	16.92%
bwt_reverse	13.41%	5.43%
readbits	12.34%	10.05%
bwt_transform	7.98%	7.98%

needbits	5.87%	1.18%
_mask	4.71%	4.71%
_more	4.69%	2.82%
_read	1.87%	1.87%

Read together, the two columns already name the targets. find_next_symbol accounts for 46.43% of runtime once its callees are included, and the bit accessors it calls: snoopbits, readbits, _mask, needbits and _more, add up to another 28% of self time on their own. move_to_front is unusual: its cumulative and self figures are identical at 16.92%, because it calls nothing. All of that time is the function itself.

3.4 Call-graph report with perf

We also recorded a call-graph profile with perf against the debug build of Python, which reports CPython's own C-level internals rather than pyflate's Python frames:

```
    perf record -F 999 -e cpu-clock --call-graph dwarf -- python3-dbg pyspy_driver.py 1
perf report --stdio --children --no-inline


# Samples: 3K of event 'cpu-clock'
# Event count (approx.): 3428428425


# Children      Self  Command      Symbol
# ........  ........  ...........  ...............................
    99.04%    23.01%  python3-dbg  [.] _PyEval_EvalFrameDefault
            |--76.03%--_PyEval_EvalFrameDefault
            |          |--74.86%--call_function.lto_priv.0
```

_PyEval_EvalFrameDefault: the CPython bytecode interpreter loop — is 23.01% self time. This is the same finding as the py-spy profile seen from the other side: the program is spending its life dispatching Python bytecodes.

3.5 The bottleneck is instruction count

Looking at the results, one metric stands above the rest: pure instruction count. This is not a problem of branch misses or cache misses, but simply of executing too many lines of code. It is mainly a result of being a pure-Python library, with no efficient C library calls. The comparison against a C bzip2 implementation makes this clear:

	Time per decompression	Instructions per output byte
C bzip2 (system)	18 ms	—
Python pyflate	1.124 s	~17,900

The measured counters confirm the hypothesos: IPC is 2.64 and the branch miss rate is 0.35%. The machine is not stalling it is simply running too many instructions. The fix is to run fewer python operations.

3.6 Where the time goes

decode_huffman_block runs for essentially the whole program. The breakdown of its runtime:

Function	% of runtime
find_next_symbol	47.22%
move_to_front	15.39%
bwt_reverse	13.50%
own lines	21.58%

3.7 Hotspot 1 — **find_next_symbol**

find_next_symbol decodes one Huffman symbol from the bitstream. It is called for every symbol, costing roughly 22,500 instructions per symbol. The original implementation:

```
def find_next_symbol(self, field, reversed=True):
    cached_length = -1
    cached = None
    for x in self.table:
        if cached_length != x.bits:
            cached = field.snoopbits(x.bits)
            cached_length = x.bits
        if (reversed and x.reverse_symbol == cached) or (not reversed and x.symbol == cached):
            field.readbits(x.bits)
            return x.code
```

What it does

The decoder's only job is to map the next bits of the stream (a codeword) to the symbol they stand for — the letter "A", or "%". For every symbol it iterates on the table from the front:

- peek x.bits bits of the stream, re-peeking only when the code length changes.
- compare them against that entry's codeword x.symbol.
- on a match, consume those bits and return x.code.

The code executes a linear search for the symbol, from scratch, for every symbol in the file.

An example of the Naïve Decode:

Step	Code length	Peeked bits	Compare	Result
entry 1	2 bits	peek 11	= 00 ?	no
entry 2	2 bits	reuse 11	= 01 ?	no
entry 3	2 bits	reuse 11	= 10 ?	no
entry 4	3 bits	peek 110	= 110 ?	MATCH

Measured on the benchmark's own input, the table holds 147 entries. An average run iterates through the loop 6 times. That means running the code 6 times for every symbol.

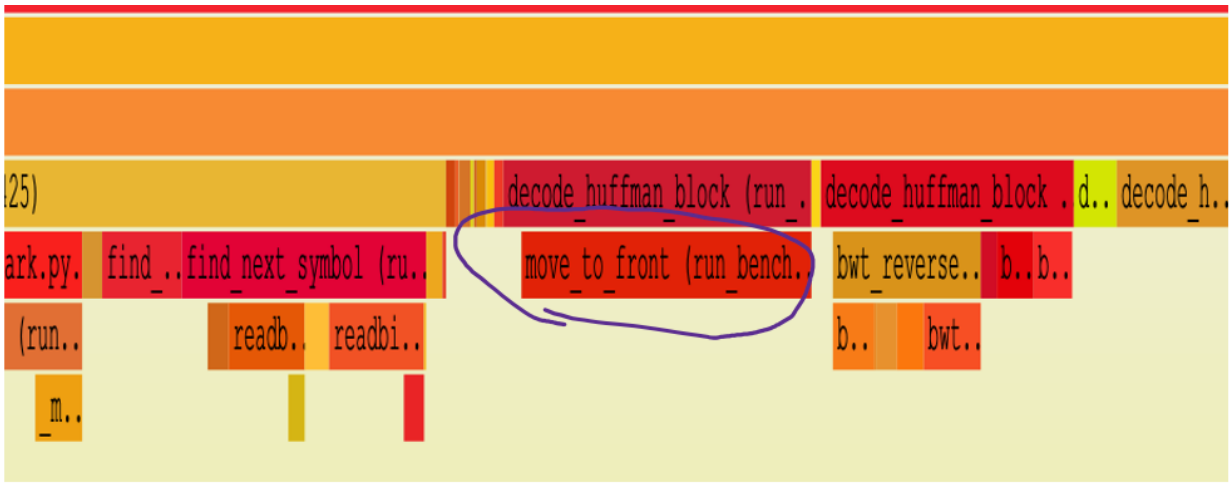
The problem

- **Linear search:** 147 entries per table.
- **No memory between calls:** nothing is cached, so each of the 148,271 symbols restarts the scan from the front.
- **Every peek is a call chain:** snoopbits() → needbits() + _mask(), then readbits() — about 9 Python calls per symbol.
- **The structure is ignored:** bzip2 organises the data in a way that makes the search unnecessary, but the implementation does not exploit it.

Cost of find_next_symbol	Value
Share of total runtime	46.43%
Instructions per symbol	~22,500
Cycles per symbol	~8,500
Loop iterations per decompression	~890k

3.8 Hotspot 2 — move_to_front

The flame graph shows a plateau on the move_to_front stage, 16.92% of the run and as noted above, all of it is self time.



The method is called on average 90,000 times per decompression, on a list of about 150 entries.

```
def move_to_front(l, c):
    l[:] = l[c:c + 1] + l[0:c] + l[c + 1:]
```

The original move_to_front.

MTF exists precisely because the data has strong locality, where the same symbol recurs immediately. The implementation ignores that locality completely and rebuilds the entire 150-element list on every call.

Additionally code is that it uses Python's Slice – which rebuilds the whole list for every call.

4. Software Optimizations

4.1 Optimization 1 — canonical Huffman decoding

Since bzip2 is a file format, it controls both the encode and the decode stage. This lets us exploit a property called **canonical Huffman**.

Canonical Huffman: the codeword is determined by the code lengths alone, through a fixed assignment rule. Only the lengths have to be stored in the file; the decoder rebuilds identical codes.

The canonical Huffman rules

- Sort the symbols by their assigned length, shortest first, and secondly by symbol index.
- Initialize the first codeword to all zeros.
- Calculate subsequent codes: if curr_length == prev_length, then new_code = prev_code + 1.
- If curr_length > prev_length, then new_code = (prev_code + 1) << (curr_length − prev_length).

The consequence is the property the optimization rests on: all codes of a given length are consecutive integers.

The improved algorithm

- Instead of looping through all table entries, we search one length at a time.
- We hold length bits as the current candidate and compare it to the largest code of that length. Because codes are ordered, a value larger than that limit means the length is wrong — take one more bit.
- Once we are at the right length, we use another property of canonical Huffman: if our code is the k-th code of that length, its value is exactly k above the base of that length. The symbol follows by a subtraction and an array lookup.
- The limit and base arrays are built once per Huffman table and used throughout the decode.

```
# ---------- 1. Build the tables: once per Huffman table ----------

def build_decode_table(self):
    limit = [-1] * (self.max_bits + 1)
    base = [0] * (self.max_bits + 1)
    perm = []
    for index, x in enumerate(self.table):
        perm.append(x.code)
        if limit[x.bits] < 0:
            base[x.bits] = x.symbol - index
        limit[x.bits] = x.symbol
    self.limit, self.base, self.perm = limit, base, perm
```

```
# ---------- 2. Decode one symbol: walk lengths, no function calls ----------

def find_next_symbol(self, field, reversed=True):
    if not reversed and self.perm is not None:
        limit = self.limit
        n = self.min_bits
        if field.bits < n:
            field.needbits(n)
        v = (field.bitfield >> (field.bits - n)) & ((1 << n) - 1)
        while v > limit[n]:                     # not a complete code yet
            n += 1                              # take one more bit
            if field.bits < n:
                field.needbits(n)
            v = (field.bitfield >> (field.bits - n)) & ((1 << n) - 1)
        remaining = field.bits - n              # consume the n bits
        field.bits = remaining
        field.bitfield &= (1 << remaining) - 1
        return self.perm[v - self.base[n]]
```

The new decode path: a walk over code lengths, with the bit accessors inlined.

Explaining the Improvement:

Instead of looping through the entire array, we can iterate by length of code. We can utilize the key feature of Canonical Huffman – the codes are sorted by value:

Example:

Length	Symbol	Code (in bits)
2	'A'	00
2	'B'	01
3	'C'	100
3	'D'	101
4	'E'	1100
4	'F'	1101

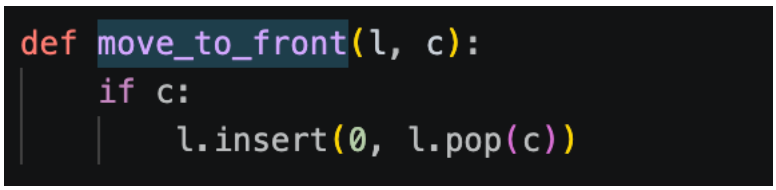
We can notice that codes go up in value. We exploit this fact in our optimization. Given the current "candidate" bits – we can compare it's value to the **largest** of that specific length. If it's larger, we know for a fact that it is not one of the symbols in that Length Group. By iterating upwards with code length, we can guarantee that the first time a code is smaller than the largest in it's group, then that it's exact length.

4.2 Removing the Python call chain

- **Original:** each loop calls snoopbits() → needbits() + _mask(), then readbits() — about 9 calls per symbol.
- **New:** the same logic is written inline in the loop, so the decode path makes no calls at all.
- **Result:** snoopbits / readbits / _mask fall from 24.18% to 0.63% of runtime.

4.3 Optimization 2 — move_to_front

- Called about 90,000 times per decompression, on a list of ~150 entries.
- The original uses Python slicing, which rebuilds the whole list on every call: three slices, concatenated into a fourth list — four O(n) passes and four allocations.
- The new version uses l.insert(0, l.pop(c)). Both are C-level list methods that shift elements inside the existing array and allocate nothing.
- The c == 0 case — the repeat case that MTF exists to make common — now returns immediately.



The optimized move_to_front.

4.4 Correctness

verify_correctness.py decompresses the benchmark's own input three ways — the pristine baseline implementation, the optimized one, and libbz2 — and requires all three to be byte-identical:

```
baseline    399360 bytes  md5=afa004a630fe072901b1d9628b960974
optimized   399360 bytes  md5=afa004a630fe072901b1d9628b960974
libbz2      399360 bytes  md5=afa004a630fe072901b1d9628b960974

all three identical: True
```

5. Performance Comparison

Both versions were measured in one scripted run, on the same VM and in the same virtual environment, with pyperformance taking 60 samples of each. The script asserts that both runs used the same interpreter before it reports anything.

5.1 Wall clock

```
+-----------+----------+-----------+--------------+-----------------------+
| Benchmark | baseline | optimized | Change       | Significance          |
+===========+==========+===========+==============+=======================+
| pyflate   | 1.14 sec | 714 ms    | 1.60x faster | Significant (t=48.58) |
+-----------+----------+-----------+--------------+-----------------------+


Mean +- std dev: 1.14 sec +- 0.07 sec -> 0.71 sec +- 0.01 sec: 1.60x faster
```

pyperformance compare — results/compare_pyflate.txt

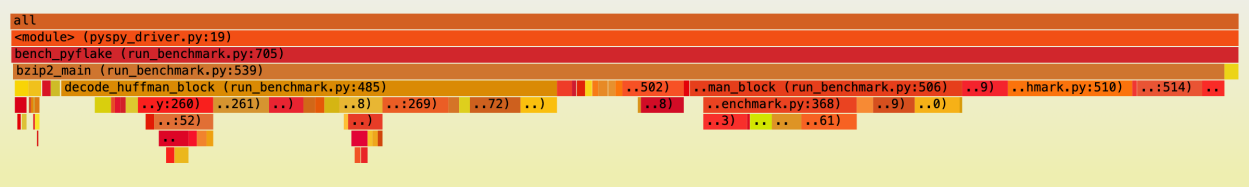
Metric	Baseline	Optimized	Change
Mean	1.141 s	0.714 s	1.60x faster
Median	1.125 s	0.712 s	1.58x faster
Time saved	—	—	427 ms per decompression

5.2 Hardware counters

Counter	Baseline	Optimized	Change
cycles	13,595,107,086	8,815,163,489	−35.2%
instructions	35,899,254,692	23,821,962,019	−33.6%
branches	7,151,658,799	4,510,978,821	−36.9%
branch-misses	24,879,633	14,211,080	−42.9%
cache-references	24,956,205	20,504,605	−17.8%
cache-misses	1,849,367	1,863,201	+0.7%
L1-dcache-loads	10,875,206,264	7,176,067,428	−34.0%
L1-dcache-load-misses	278,024,538	70,306,891	−74.7%
IPC	2.64	2.70	+2.3%

- Instructions fell 33.6% and cycles by 35.2%, while IPC barely moved.
- The large drop in L1 data-cache misses comes from the move_to_front change, which stopped allocating four fresh lists on every call.
- The last-level cache miss rate rises (7.41% → 9.09%) even though the absolute miss count is flat — cheap hits were removed, so the ratio grows. Nothing became slower.

5.3 The flame graph after optimization



Comparing against Figure 1: the call towers beneath find_next_symbol have disappeared, because the bit accessors are now inline arithmetic rather than Python calls. What remains wide is bwt_transform and bwt_reverse, which we did not touch.

5.4 Where the time went

Function	Baseline	Optimized
find_next_symbol	46.43%	24.07%
move_to_front	16.92%	2.17%
snoopbits / readbits / _mask	24.18%	0.63%
bwt_reverse	13.41%	13.59%
bwt_transform	7.98%	7.86%

5.5 Each change measured on its own

To confirm that neither optimization double-counts the other, we built one variant per change from the same sources and measured each separately. Every variant asserts the output md5, so a broken variant cannot produce a timing:

```
    run_benchmark.py.baseline best=1.1261s  (md5 ok)
    instructions: 21524503431
v_mtf_only.py             best=0.9491s  (md5 ok)
    instructions: 19045898811
v_huff_only.py            best=0.8691s  (md5 ok)
    instructions: 16979507191
run_benchmark.py          best=0.7099s  (md5 ok)
    instructions: 14344547696
```

The results:

Variant	Time	Change	Instructions
original	1.126 s	—	21.52 B
move_to_front only	0.949 s	−15.7%	19.05 B
find_next_symbol only	0.869 s	−22.8%	16.98 B
both	0.710 s	−37.0%	14.34 B

The two optimizations are additive to within 4.3% on time and 2.2% on instructions, so neither number double-counts the other.

5.6 Per decoded symbol

	Baseline	Optimized
instructions in find_next_symbol	~22,500	~12,400
cycles in find_next_symbol	~8,500	~4,600
steps to decode one symbol	6.0 entries	2.3 lengths

5.7 Measured improvement per optimization

Optimization 1 — canonical Huffman decoding

- 6.0 table entries → 2.3 length steps per symbol.
- 48.1% of symbols decode on the first peek.
- Worst case 14 steps over code lengths, not 147 table entries.
- find_next_symbol: 46.4% → 24.1% of runtime.
- ~22,500 → ~12,400 instructions per decoded symbol.

Optimization 2 — move_to_front

- move_to_front: 16.9% → 2.2% of runtime.
- This change on its own: 15.7% faster.
- 4 allocations per call → 0.

7. Conclusion

The goal was to make pyflate faster without changing a single byte of its output. The profiling gave a clear answer about where to look. IPC was 2.64 and only 0.35% of branches missed, so the processor was not stalling on memory and it was not mispredicting. It was simply being handed far too many instructions, which makes sense for a pure Python decompression algorithm.

The “obvious answer” would just be to rewrite with C, but we must remember that the purpose of the benchmark is to benchmark different versions of Python. Nevertheless, we found a way to improve the runtime of the benchmark by optimizing the actual algorithms and data structures that it uses.

Two changes came out of that. In find_next_symbol we dropped the linear scan over the Huffman table and used a new iteration method, which works because bzip2 hands out canonical Huffman codes. In move_to_front we replaced the slice rebuild with a pop and an insert. Both functions do exactly what they did before. The speedup comes from structure that bzip2 already provides and that the original code ignored.

Together the two changes took the benchmark from 1.141 s to 0.714 s, a 1.60x speedup, with 33.6% fewer instructions. Measured on their own, the Huffman change is worth 22.8% and move_to_front 15.7%, and both together 37.0%.

What is left is the BWT. bwt_transform and bwt_reverse were never touched, and once the other two hotspots shrank they grew to about a third of the runtime. A third software optimization would go there.

Appendix A — Raw data

Every figure and table in this report is generated by Pyflate/script_pyflate.sh and stored in Pyflate/results/:

File	Contents
compare_pyflate.txt	pyperformance compare output and the perf stat counter table
ablation.txt	per-variant timings and instruction counts
verify_correctness.txt	md5 agreement of baseline, optimized and libbz2
script_pyflate.log	full log of the scripted run
baseline/pyflate_flamegraph_baseline.svg	Figure 1, interactive version
baseline/pyflate_hotspots_baseline.txt	self, cumulative and per-function tables
baseline/pyflate_perfstat_baseline.txt	hardware counters, three event groups
baseline/perf_report_baseline.txt	perf call-graph report
baseline/pyflate_baseline.json	raw pyperformance samples
optimized/*	the same set of artefacts for the optimized build